

Professional Self-Assessment

By John Grudzinski

My ePortfolio represents the growth I have made throughout my Computer Science journey at Southern New Hampshire University and demonstrates my preparation for a career as a secure full stack software developer. When I first entered the program, my main goal was to learn how to build software and understand the foundations of computer science. As I progressed through the program, I became more focused on that goal. My interest increased in how front-end interfaces, back-end logic, databases, security practices, testing, and user needs work together to create fully complete software systems. Overall, this capstone allowed me to bring those skills together by revisiting my earlier projects, evaluating their weaknesses, and improving them into more professional artifacts to reflect my progress as a developer.

Throughout the program, I developed a stronger understanding of software development as an iterative and problem-solving process. In the early days of my coursework, my main focus was often on making sure a program worked and met the immediate assignment requirements. As I gained more experience, I learned to think more like a software developer by considering readability, maintainability, testing, security, user needs, and future updates. This shift helped shape my professional goals and values. My desire is to build software that is not only functional, but also reliable, secure, understandable, and helpful to the people who depend on it.

One of the most important skills I worked on was my communication. I learned firsthand that in computer science, technical ability is important, but it is not enough by itself. As a developer I also need to explain technical decisions clearly to my teammates, managers, clients, and other stakeholders who may not possess the same level of technical knowledge. Throughout the program, I improved this skill through project documentation, written reflections, code comments, code review preparation, and narrative writings. These activities helped me learn how to explain what a program does, why a design decision matters, what tradeoffs exist, and how a solution supports a larger goal. This skill is especially important for my chosen career path as full stack developer because my work will often connect multiple audiences, including users, developers, database administrators, testers, and decision makers.

Another important area of growth for me is collaboration. Although all my course projects were completed individually, the program still helped me instill good habits that support team-based development. Code review, readable code, clear comments, organized project files, and structured documentation all make it easier for other people to understand and maintain a project. In a professional environment, a developer rarely writes code only for

themselves. The code must be understandable to future developers, testers, project managers, and stakeholders. Through the capstone code review process, I practiced evaluating code in a way that was constructive and focused on improvement rather than simply pointing out mistakes. This experience helped me better understand how code review supports collaboration, quality assurance, and shared decision making.

My ability to communicate with stakeholders has also been strengthened by my coursework because I learned to connect my technical work to practical needs. Instead of viewing assignments as isolated programming tasks, I learned to think about the problem the software is supposed to solve. A dashboard is not only a database project but also a decision-making tool. A mobile application is not only an interface, but also a tool that helps a user complete tasks quickly and accurately. A testing project is not only about passing tests, it is also about proving that software meets requirements and behaves correctly under expected and unexpected conditions. This perspective helped me become more aware of how requirements, usability, data, testing, and security all affect the final product.

A major part of my growth came from learning how to use data structures and algorithms to solve problems more effectively. Through my programming courses, I learned that the way data is stored and accessed can significantly affect the clarity and efficiency of a program. Lists, sets, maps, loops, conditionals, filtering, sorting, and validation all have practical uses depending on the problem being solved. I also learned that design choices involve tradeoffs. For instance, a simple list may be easy to understand, but a map may be better for faster lookup. A reusable helper method may take more planning at first, but it can also reduce repeated logic and make the program easier to maintain. These lessons helped me think beyond basic functionality and consider whether the structure of a solution fits the problem at hand.

Software engineering and database work have also become central to my career interests. Throughout the program, I gained experience with object-oriented programming, mobile application development, local storage, MongoDB, CRUD operations, dashboard development, as well as database driven applications. I learned that software engineering is about organizing a system so it can grow and be maintained, while database work is about storing, retrieving, filtering, and protecting information in a useful way. A full stack developer needs both of these skill sets because modern applications often depend on a connection between user interfaces, application logic, and persistent data. My coursework helped me understand how these pieces fit together and why a strong design matters before, during, and after implementation.

Testing and quality assurance became especially important to me in CS 320, Software Testing, Automation, and Quality Assurance. In that course, I worked with contact, task, and appointment services for a mobile application back end. The services used in-memory data structures and were tested with JUnit to verify that the software met specific customer requirements. For example, the contact service required fields such as contact ID, first name, last name, phone number, and address to follow length and null restrictions. The task service required unique task IDs, valid names, valid descriptions, and update/delete behavior. The appointment service required valid appointment IDs, future appointment dates, descriptions, and delete functionality. Writing JUnit tests for these requirements helped me understand that good testing is not random; it should be tied directly to the requirements the software is expected to meet.

A specific example from the program outside of my enhanced artifacts is CS 320 which also helped me develop a more disciplined testing mindset. I practiced testing valid inputs, invalid inputs, boundary conditions, null values, duplicate IDs, update behavior, delete behavior, and exception handling. This taught me that software should not be considered complete just because it runs once under ideal conditions. It should be tested against the ways users might make mistakes, the ways data might be invalid, and the ways requirements might be violated. The course also helped me understand the importance of limiting bias when testing my own code. As a developer, it is easy to assume the code works because I know what I intended it to do. However, as a tester, I had to look at the code more critically and ask whether it actually met the requirements. That mindset connects directly to my goal of being a secure full stack developer because reliable systems depend on validation, testing, and disciplined attention to quality.

One of the strongest themes in my professional growth has been security. In the early days of my coursework, I understood security mostly as a separate topic. As I progressed, I began to see that security should be part of the entire development process. Input validation, password protection, database credentials, user permissions, error handling, testing, and data privacy are all design concerns, not just final checks. A secure mindset means asking how a system could fail, how data could be exposed, and how a user or attacker might interact with the software in unexpected ways. This mindset is important to me because I want to pursue secure full stack development, where both the front end and back end must be designed with trust, privacy, and resilience in mind.

Developing this ePortfolio helped me showcase my strengths and prepare for the computer science field because it required me to organize my work as a professional body of evidence. Instead of only showing final code, I had to explain my process, describe my improvements, and connect my work to course outcomes. This helped me understand how

to present myself to my future employers. My portfolio shows that I can evaluate existing work, improve it, explain my choices, and reflect on what I learned. It also shows that I can work across multiple areas of computer science, including software engineering, algorithms and data structures, databases, communication, testing, and security.

The artifacts included in this portfolio fit together to demonstrate the range of skills I developed in the program. The software design and engineering artifact shows my ability to improve application structure, authentication, validation, maintainability, and user specific data handling. The algorithms and data structures artifact shows my ability to improve logic, lookup efficiency, filtering, sorting, validation, and organization through better data structure choices. The database artifact shows my ability to improve data access, database security, query design, aggregation, projection, indexing, and dashboard reliability. Together, these artifacts demonstrate that I can work across the layers of a software system rather than focusing on only one narrow part of development.

The code review video serves as the foundation for my portfolio because it shows how I approached the enhancement process. In the review, I examined existing functionality, identified areas for improvement, and explained how each planned enhancement would support the three major computer science categories. My written enhancement narratives then provide written explanations of why I selected each artifact, how it was improved, what skills I demonstrated, and how my work aligns with the course outcomes. The original and enhanced code files provide evidence of my actual technical growth. Together, the code review, artifacts, narratives, and this self-assessment create a complete picture of my development as a computer science student and emerging software professional.

Overall, my ePortfolio demonstrates my progress towards the five computer science program outcomes. I have shown strategies that support collaboration through code review, readable comments, organized files, and documentation. I have demonstrated professional communication through written narratives, the code review presentation, and my GitHub Pages portfolio. I have applied algorithmic principles through improved data structures, filtering, sorting, validation, and logic design. I have used well known tools and techniques such as Java, Android Studio, SQLite, Python, MongoDB, Dash, Plotly, JUnit, testing practices, and GitHub Pages to build and improve software. I have also demonstrated that I have a security mindset by addressing password storage, user specific data access, input validation, credential handling, defensive database operations, and requirement-based testing.

Completing this capstone has shown me how much I have grown since entering the Computer Science program. I am now better prepared to evaluate existing code, improve software design, communicate technical decisions, work with data, test for reliability, and

think about security throughout the development process. My portfolio is not only a final course project but also a professional representation of the skills I can bring into the software development field.